
El nodo como flecha

Un modelo de tres coordenadas para el diseño
de un catálogo de nodos componible

Familias, cardinalidad y mundos de datos

Documento técnico

Motor de automatización de flujos de trabajo

Mauricio Daniel Klainbard

18 de agosto de 2026

Documento de carácter metodológico. Presenta un modelo conceptual para el diseño de nodos y su clasificación; es una arquitectura de razonamiento, no una especificación de implementación, y no altera los nodos del catálogo actual.

Resumen

Un catálogo de nodos de automatización se diseña, habitualmente, nodo por nodo y de forma aislada. Este documento sostiene que el diseño aislado es la causa raíz de una patología recurrente: nodos que, al integrarse al ecosistema, exhiben pocas interacciones útiles con el resto y obligan a rediseños sucesivos. Como remedio se propone un *modelo de diseño* que reemplaza la pregunta “¿qué hace este nodo?” por “¿cómo se compone este nodo con los demás?”. El punto de partida es un intento fallido pero fructífero: definir las *familias* de nodos como clases de equivalencia de una relación de no-conexión. Se muestra que dicha relación es reflexiva y simétrica pero *no transitiva*, por lo que no induce una partición. El análisis de por qué falla conduce al modelo central: un nodo no es un punto sino una *flecha* entre *mundos de datos*, y su lugar en el ecosistema queda fijado por tres coordenadas —la firma de mundo, que fija a la vez su origen y su destino; la cardinalidad; y la aridez de puertos que describe su ramificación—. Se demuestra que este esquema recupera la intuición original de familia como un teorema en lugar de una definición, clasifica correctamente los nodos de control de flujo (**if**, **switch**, **merge**) y, sobre todo, ofrece una disciplina de diseño operativa. Se argumenta finalmente que las tres coordenadas constituyen una *gramática composicional* que eleva de forma notable la calidad de los flujos que un asistente de lenguaje —incluso uno pequeño— puede construir. Se añade, por último, una cuarta coordenada de naturaleza distinta —el *carril de transporte*, que distingue el envío por filas del envío columnar (Arrow)—: a diferencia de las tres primeras, que son advisorias, su frontera es una *cuestión de seguridad*, pues el cruce entre carriles materializa un lote acotado en un número arbitrariamente grande de elementos y, sin defensa, un solo nodo bastaría para agotar el motor.

Palabras clave: diseño de nodos, composicionalidad, relaciones de equivalencia, cardinalidad, tipos de datos, automatización de flujos de trabajo, asistentes de lenguaje.

Índice

1. La dificultad de diseñar un nodo	3
1.1. El diseño aislado y su patología	3
1.2. Una metodología insuficiente: un nodo por día	3
2. Un primer intento: las familias como clases de equivalencia	3
2.1. La intuición de familia	3
2.2. La relación de no-conexión	4
2.3. El fracaso: la transitividad	4
3. El modelo de tres coordenadas: el nodo como flecha	5
3.1. El nodo como flecha entre mundos de datos	5
3.2. La primera coordenada y su insuficiencia	6
3.3. La segunda coordenada: la cardinalidad	7

4. El modelo frente a if y switch	8
4.1. En el eje de mundos: son la identidad	8
4.2. En el eje de cardinalidad: una partición conservativa	8
4.3. La coordenada que esto fuerza, y su dual	8
5. Los mundos de datos y la disciplina de diseño	9
5.1. Qué es un mundo	9
5.2. Por qué los mundos guían el diseño	10
6. La cuarta coordenada: el carril de transporte	11
6.1. Dos carriles para un mismo mundo	11
6.2. El puente entre carriles no es gratis	11
6.3. Por qué es una frontera de seguridad y no una optimización	11
6.4. La disciplina de diseño que impone	12
7. Consecuencia: un asistente de IA que compone mejor	13
7.1. De la generación libre a la búsqueda tipada	13
7.2. Por qué basta un modelo pequeño	13
8. Conclusiones	14

1 La dificultad de diseñar un nodo

1.1 El diseño aislado y su patología

Diseñar un nodo parece, a primera vista, un problema local: se elige una operación útil —renderizar un gráfico, resolver una ecuación diferencial, leer una hoja de cálculo—, se definen sus parámetros de configuración y su lógica, y se lo agrega al catálogo. Esta forma de proceder, que llamaremos *diseño aislado*, trata cada nodo como una unidad autocontenida cuya bondad se juzga por lo bien que realiza su tarea.

El diseño aislado falla por una razón estructural: un nodo no vive solo. Su valor en una plataforma de automatización no reside en lo que hace por sí mismo, sino en *con qué otros nodos se compone*. Un nodo excelente en aislamiento pero que no recibe datos de ninguna fuente natural, o cuya salida ningún otro nodo puede consumir, es un nodo pobre: una isla en el grafo. Y esta propiedad —la riqueza de sus interacciones— es justamente la que el diseño aislado no contempla, porque mira el nodo y no el ecosistema.

La consecuencia práctica es un ciclo de retrabajo. Un nodo concebido de forma aislada se implementa, se intenta insertar en flujos reales, y solo entonces se descubre que su forma de entrada no coincide con lo que producen los nodos que deberían precederlo, o que su salida obliga a intercalar adaptadores, o que su cardinalidad hace estallar la tubería. Se lo rediseña; el rediseño revela una nueva fricción; se lo vuelve a rediseñar. El costo no está en escribir el nodo sino en *descubrir, tarde y empíricamente, cómo debería haber sido su forma*.

1.2 Una metodología insuficiente: un nodo por día

Una respuesta natural al problema es organizativa: fijar un ritmo —por ejemplo, un nodo por día— y avanzar. Pero el ritmo no ataca la causa. Diseñar un nodo por día *sigue siendo* diseño aislado, solo que calendarizado; cada nodo se concibe sin un marco que lo obligue a declarar, de entrada, cómo encaja con los demás. El retrabajo no desaparece: se difiere y se acumula.

Lo que hace falta no es una cadencia sino un *modelo*: un lenguaje pequeño y preciso en el que la pregunta “¿cómo se conecta este nodo con el resto del ecosistema?” se pueda responder *antes* de escribir una sola línea, y en el que un mal diseño —una isla— sea visible como tal en el papel. El resto de este documento construye ese modelo. Es deliberadamente una arquitectura de razonamiento, externa al código: no introduce propiedades nuevas en los nodos ni mecanismos de verificación en el motor, y no altera en nada el catálogo actual. Su único propósito es enseñarnos a diseñar.

2 Un primer intento: las familias como clases de equivalencia

2.1 La intuición de familia

La primera observación fructífera es que los nodos se agrupan en *familias* definidas por el ecosistema sobre el que operan: la familia de los gráficos (CHART), la de las integraciones con un proveedor (GOOGLE), la de los métodos numéricos. Se buscó dar a esta noción intuitiva

una base formal. La herramienta evidente, tomada de la teoría de conjuntos, es la *relación de equivalencia*: una relación que sea reflexiva, simétrica y transitiva induce una partición del conjunto en clases disjuntas [1], y esas clases serían, precisamente, las familias.

2.2 La relación de no-conexión

Como diseñábamos pensando en la interacción entre familias, se propuso definir la relación por su *ausencia*. La observación de partida fue que dos nodos de la familia CHART no pueden conectarse entre sí: todos producen una imagen a partir de datos numéricos, y ninguno consume una imagen ni produce datos numéricos, de modo que la salida de uno nunca alimenta la entrada de otro.

Definición 2.1 (Relación de no-conexión). Sean a, b dos nodos. Se dice que $a \perp b$ si y solo si no existe conexión posible entre a y b en ninguna de las dos direcciones; es decir, ni la salida de a puede alimentar significativamente la entrada de b , ni la de b la de a .

La idea era sembrar cada familia con un nodo representativo r y definir su clase como $\{x : x \perp r\}$. Examinemos si $\perp$ es una relación de equivalencia.

Simetría. Es inmediata por construcción. La cláusula “en ninguna de las dos direcciones” es indiferente al orden de a y b ; luego $a \perp b \iff b \perp a$.

Reflexividad. A primera vista falla. Un nodo **filter** puede conectarse con otro **filter** —la salida del primero es una entrada válida para el segundo—, de modo que $\neg(\text{filter} \perp \text{filter})$. Sin embargo, dos nodos de la misma especie encadenados son *reducibles* a uno solo: dos filtros en serie equivalen a un único filtro con la conjunción de ambas condiciones. Bajo la convención de que dos instancias reducibles se identifican, la autoconexión no aporta un grafo genuinamente nuevo, y podemos adoptar $a \perp a$ por convención. La reflexividad se sostiene, aunque como estipulación y no como hecho.

2.3 El fracaso: la transitividad

La propiedad decisiva —y la que hunde el modelo— es la transitividad. Para que $\perp$ particione, debería cumplirse

$$(a \perp b) \wedge (b \perp c) \implies (a \perp c).$$

No se cumple. La razón profunda es que “no conectar” es la *negación* de una relación de compatibilidad que es a la vez *dirigida* (la salida de uno contra la entrada del otro) y de *dos ejes* (qué produce cada nodo, qué consume cada nodo); la negación de una relación con esa estructura no es transitiva en general. Un contraejemplo mínimo lo exhibe.

Ejemplo 2.1 (Ruptura de la transitividad). Considérense tres nodos:

- A : una *fuentes numérica* (produce números, no consume nada);
- B : una *fuentes de imágenes* (produce una imagen, no consume nada);
- C : un **chart** (consume números, produce una imagen).

Entonces:

- $A \perp B$: dos fuentes, ninguna consume la salida de la otra. ✓
- $B \perp C$: C consume números y B produce una imagen (incompatible); B no consume nada. ✓
- $\neg(A \perp C)$: A produce números y C los consume, luego $A \rightarrow C$ es una conexión legítima. ✗

Se tiene $A \perp B$ y $B \perp C$ pero $\neg(A \perp C)$: la transitividad falla.

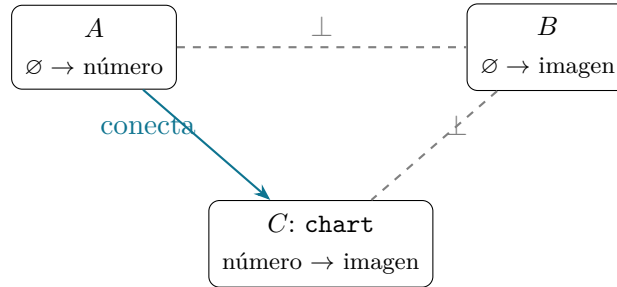


Figura 1: El contraejemplo del Ejemplo 2.1. Las aristas punteadas indican no-conexión ($\perp$); la flecha llena, una conexión legítima. $A \perp B$ y $B \perp C$ no fuerzan $A \perp C$.

Hay, además, una segunda grieta que revela que $\perp$ ni siquiera captura la noción que pretendía. La familia CHART es, en efecto, cerrada bajo $\perp$ (sus miembros no se conectan entre sí), pero la familia GOOGLE no lo es: leer un correo y escribir en una hoja de cálculo son ambos nodos de GOOGLE y *sí* se conectan. La relación $\perp$ describe un *tipo* de familia —la conversora— y no otro —la de proveedor—. La misma causa (la compresión de una estructura dirigida y de dos ejes en una relación simétrica de un solo eje) produce las dos fallas.

Observación 2.1. *El intento no fue estéril. Exigirle a la noción de familia las tres propiedades de una relación de equivalencia —la disciplina que Hrbacek y Jech [1] imponen a toda partición— es exactamente lo que expuso que $\perp$ era el objeto equivocado. Y la teoría de conjuntos ofrece también el remedio: toda función induce una relación de equivalencia canónica —su núcleo, $a \sim b \iff f(a) = f(b)$ —, cuyas clases son las fibras de la función y particionan sin excepción. El modelo que sigue no abandona la idea de equivalencia: la funda sobre la función correcta.*

3 El modelo de tres coordenadas: el nodo como flecha

3.1 El nodo como flecha entre mundos de datos

El error de fondo del primer intento fue tratar el nodo como un *punto* que se relaciona con otros puntos. La corrección es tratarlo como una *flecha*: un nodo transporta datos de un *mundo* a otro.

Definición 3.1 (Nodo como flecha). Sea $\mathcal{W}$ el conjunto de *mundos de datos* (la forma semántica que puede tener aquello que fluye: una tabla de registros, un texto, una imagen, un archivo, la nada, etc.; la Sección 5 los desarrolla). Un nodo f es una flecha

$$f : X \longrightarrow Y, \quad X, Y \in \mathcal{W},$$

donde $X = \text{src}(f)$ es su mundo de origen (lo que consume) y $Y = \text{dst}(f)$ su mundo de destino (lo que produce).

$$\begin{array}{ccc} X & \xrightarrow{\quad f \quad} & Y \\ & \text{origen} \rightarrow \text{destino} & \end{array}$$

Figura 2: La unidad del modelo: un nodo es una flecha de un mundo de origen a un mundo de destino.

Bajo esta lente, la noción de conexión se vuelve precisa y recupera lo que $\perp$ buscaba a tientas.

Definición 3.2 (Conexión componible). Dos nodos a, b se conectan, $a \rightarrow b$, de forma significativa si y solo si $\text{dst}(a) = \text{src}(b)$: la salida de a vive en el mismo mundo que la entrada de b . La conexión es, literalmente, la *composición de flechas*.

Esto es fiel a la naturaleza del motor, cuyo cable es *no tipado*: por él viaja siempre un elemento con una forma libre, y el editor jamás prohíbe una arista. La compatibilidad no la impone el grafo; la impone la *forma semántica* —el mundo— de lo que la flecha produce frente a lo que la siguiente consume. El modelo no describe una restricción del motor, sino la disciplina con la que *nosotros* razonamos sobre él.

3.2 La primera coordenada y su insuficiencia

Con las flechas aparece de inmediato la relación de equivalencia correcta, como núcleo de una función.

Definición 3.3 (Firma de mundo). La *firma de mundo* de un nodo es el par $\sigma(f) = (\text{src}(f), \text{dst}(f))$. Dos nodos son de la misma familia-de-mundo cuando $\sigma(a) = \sigma(b)$.

Por ser el núcleo de σ , esta relación es reflexiva, simétrica y transitiva *por construcción*: particiona sin colapsar, y el escenario de colapso que temíamos —un nodo que arrastra dos familias a fusionarse— es imposible, porque las fibras de una función nunca se solapan. Además, la propiedad que $\perp$ quería *definir* reaparece aquí como un *teorema*: una familia con $\text{src} \neq \text{dst}$ (fuera de la diagonal) es automáticamente cerrada bajo no-conexión —dos flechas $X \rightarrow Y$ con $X \neq Y$ no componen, pues requeriría $Y = X$ —. La familia CHART, toda ella $\text{TABLA} \rightarrow \text{IMAGEN}$, lo satisface. La familia GOOGLE, que abarca varias firmas de mundo, no; y esa es exactamente la razón por la que sus nodos sí interactúan.

Pero una sola coordenada es *insuficiente*, y el propio catálogo lo delata. Considérense tres operaciones sobre tablas:

$$\text{sort} : \text{TABLA} \rightarrow \text{TABLA}, \quad \text{filter} : \text{TABLA} \rightarrow \text{TABLA}, \quad \text{reduce} : \text{TABLA} \rightarrow \text{TABLA}.$$

Las tres comparten firma de mundo: caen en la misma clase, la *diagonal* $\text{TABLA} \rightarrow \text{TABLA}$. La firma de mundo sugiere entonces que son la misma familia y —peor— que podrían reducirse a un solo nodo, exactamente como dos filtros en serie se reducían a uno. Y sin embargo no lo

son: ordenar, filtrar y plegar son operaciones distintas que no se sustituyen entre sí. Falta una coordenada que las separe.

3.3 La segunda coordenada: la cardinalidad

Lo que distingue a **sort**, **filter** y **reduce** no es a dónde llevan el dato, sino *cómo transforman la cantidad de elementos*. Esta es la segunda coordenada, y es de una clase especialmente sólida: no es vocabulario elegido, sino una invariante estructural que se lee de la operación misma.

Definición 3.4 (Cardinalidad). La *cardinalidad* $\kappa(f)$ de un nodo describe la relación entre el número N de elementos que entran y el número M que salen. Las clases relevantes son:

- **preserva** ($N:N$): un elemento entra, un elemento sale (**sort**, **set**);
- **contrae selectivo** ($N:M$, $M \leq N$): descarta elementos (**filter**);
- **contrae total** ($N:1$): pliega el lote en un resultado (**reduce**);
- **expande** ($N:M$, $M \geq N$): un elemento genera varios (**split_out**);
- **fuelle** ($0:N$) y **sumidero** ($N:0$): nace o muere el stream.

Nodo	Firma de mundo	Cardinalidad	Rol
sort	TABLA $\rightarrow$ TABLA	$N:N$	preserva (permuta)
set	TABLA $\rightarrow$ TABLA	$N:N$	preserva (transforma por ítem)
filter	TABLA $\rightarrow$ TABLA	$N:M$, $M \leq N$	contrae selectivo
split_out	TABLA $\rightarrow$ TABLA	$N:M$, $M \geq N$	expande
reduce	TABLA $\rightarrow$ TABLA	$N:1$	contrae total

Cuadro 1: La cardinalidad separa la diagonal TABLA $\rightarrow$ TABLA que la firma de mundo confundía. (Persiste un residuo: **sort** y **set** siguen compartiendo $N:N$; los separa una distinción ulterior, la de transformar el orden frente al contenido. La cardinalidad es necesaria, no siempre suficiente: es una coordenada más, no la última.)

La cardinalidad no solo refina la clasificación: *compone*, y es ahí donde se vuelve una herramienta de diseño y no una mera etiqueta. Si se piensa la cardinalidad como un multiplicador sobre el tamaño del lote —contrae (< 1), preserva ($= 1$), expande (> 1)—, al encadenar nodos los multiplicadores se multiplican:

$$\text{preserva} \circ X = X, \quad \text{expande} \circ \text{expande} = \text{expande}, \quad \text{expande seguida de } (N:1) = 1.$$

De aquí se leen tres consecuencias de diseño. Primera: dos expansores en serie son la firma de una *explosión* combinatoria del lote; el modelo la anticipa en el papel. Segunda: la reflexividad-por-reducibilidad del intento inicial se precisa —se sostiene *dentro de una misma clase de cardinalidad* (**sort** $\circ$ **sort** = **sort**, **filter** $\circ$ **filter** = **filter**), pero **reduce** $\circ$ **reduce** ya es degenerado—. Tercera: aparece un *arco de tubería* canónico, *expandir* $\rightarrow$ *preservar*^{*} $\rightarrow$ *contraer*, que es la forma de todo pipeline sano (Figura 3).

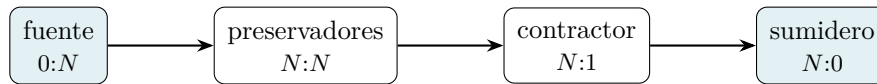


Figura 3: El estrato de tubería que induce la cardinalidad, ortogonal al eje de mundos: las fuentes abren trabajo, los preservadores lo transforman sin cambiar el conteo, los contractores lo pliegan, los sumideros lo consumen.

4 El modelo frente a `if` y `switch`

Los nodos de control de flujo son la prueba de fuego del modelo, porque no son una sola flecha: `if` tiene dos puertos de salida y `switch` tiene puertos dinámicos. Lejos de romper el esquema, obligan a explicitar la tercera coordenada —la aridez de puertos— que el resto de los nodos usaba de forma trivial.

4.1 En el eje de mundos: son la identidad

Lo primero es que `if` y `switch` *no tocan el elemento*: entra una tabla, sale la misma tabla, sin transformar, por cada puerto. Son enrutadores puros. En la firma de mundo son endomorfismos identidad $TABLA \rightarrow TABLA$, como `sort`. No mueven el dato a otro mundo; lo mueven a otro *cable*.

4.2 En el eje de cardinalidad: una partición conservativa

Como caja negra son $N:N$: no crean ni destruyen elementos. Pero ese N se *reparte* entre k puertos de salida,

$$N = n_1 + n_2 + \cdots + n_k, \quad n_i \geq 0,$$

y cada puerto, mirado por separado, es $N:n_i$ con $n_i \leq N$: es decir, *un filter*. De aquí la unificación que confirma que el modelo es el correcto:

`switch` es un banco de `filter` en paralelo que particiona el lote; `if` son dos `filter` complementarios; `filter` es un `if` al que se le descartó el puerto negativo.

`filter`, `if` y `switch` son la *misma operación* a distinta aridez de puertos (1 que se conserva, 2, k). Lo único que cambia es qué se hace con el resultado del predicado: descartarlo, rutearlo a dos puertos, o a k (con un puerto por defecto que hace de la partición una función total sobre el elemento).

4.3 La coordenada que esto fuerza, y su dual

Si solo tuviéramos las dos coordenadas anteriores, `if/switch` colapsarían con `sort` (todos $N:N$, $TABLA \rightarrow TABLA$). Lo que los separa es la *aridez de puertos*: single-port frente a multi-port. Y esta coordenada tiene un dual exacto que cierra el modelo con elegancia: `merge` es el co-enrutador — k puertos de entrada $\rightarrow$ uno de salida, unión en lugar de partición—. `switch` y `merge` son las dos caras de una misma moneda: abrir el flujo por predicado y cerrarlo por estrategia.

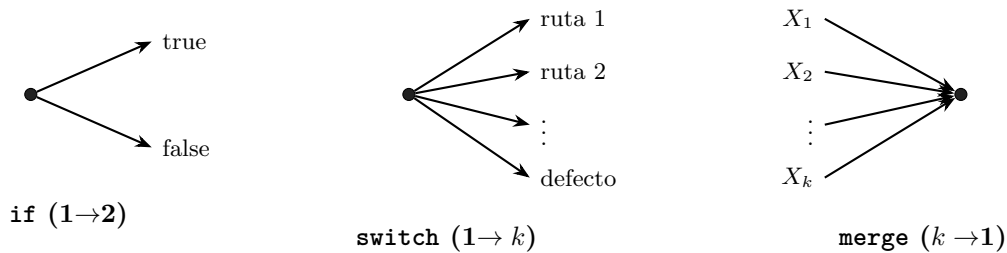


Figura 4: La aridad de puertos como tercera coordenada estructural. **if** y **switch** ramifican (una entrada, k salidas, partición conservativa); **merge** es su dual exacto (fusiona k entradas en una salida, por unión).

La familia de control de flujo queda así definida con una firma nítida y principista: *identidad en el mundo*, *multi-puerto*, y *cardinalidad conservativa* (partición al ramificar, unión al fusionar). Son los nodos que reconfiguran la *topología* del flujo sin tocar jamás el dato —y eso es lo que los distingue de la diagonal de transformación, que es single-port—. Los puertos dinámicos de **switch** no son una excepción: su aridad k es un parámetro de configuración, el mismo patrón “configuración $\rightarrow$ forma” que ya gobierna la cardinalidad de otros nodos.

Se observa el patrón que estructura todo el modelo: *cada coordenada resuelve un colapso que la anterior no podía*. La firma de mundo separó **CHART** de **filter**, pero fundió **sort/filter/reduce**; la cardinalidad los separó, pero fundió **sort** con **if**; la aridad de puertos separa el enrutador del transformador. **if** y **switch** no son una excepción al modelo: son lo que lo completa.

5 Los mundos de datos y la disciplina de diseño

5.1 Qué es un mundo

La primera coordenada descansa sobre el conjunto $\mathcal{W}$ de mundos de datos. Un mundo es la *forma semántica* de lo que fluye, no su serialización ni su tipo en el motor. Es un vocabulario que vive en la cabeza del diseñador; no se declara en ningún archivo ni altera nodo alguno. Un catálogo de trabajo usa unos pocos:

- **TABLA** — una colección de elementos con campos. Es el mundo central, el gran concentrador: casi todo produce o consume tablas.
- **TEXTO** — una cadena sin estructura impuesta; aquí viven el análisis de CSV/JSON y las expresiones regulares.
- **ARCHIVO** — bytes opacos referenciados (un adjunto); mundo de entrada de los nodos de documentos y visión.
- **IMAGEN** y **DOCUMENTO** — artefactos renderizados (un PNG, un PDF); mundos casi terminales.
- **ESCALAR** — un valor único, el resultado de un pliegue.

- RECURSO — un asa a un objeto de un proveedor (una fila de una hoja de Google, un mensaje de Slack).
- VACÍO ($\emptyset$) — la nada: el origen de las fuentes ($0:N$) y el destino de los sumideros ($N:0$).

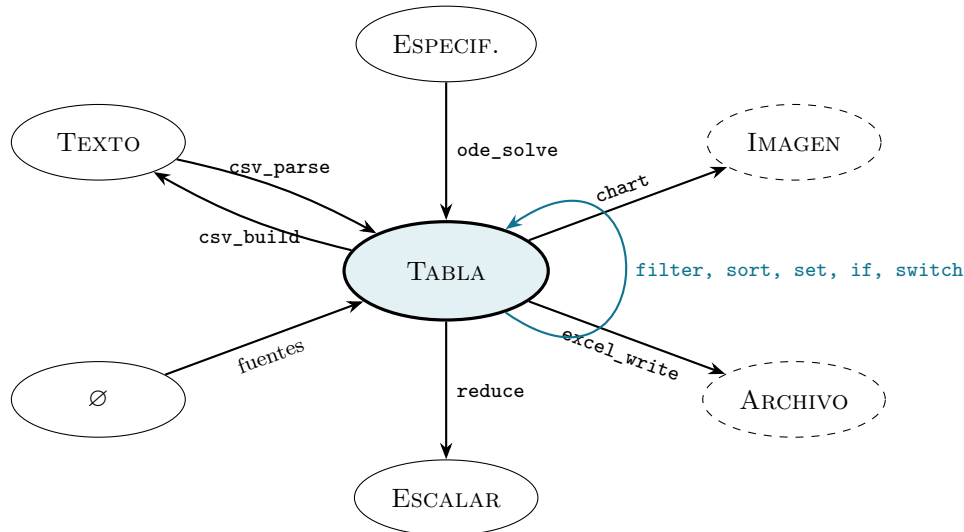


Figura 5: El mapa de mundos. Los nodos son las flechas. TABLA es el concentrador (muchas entradas y salidas); IMAGEN y ARCHIVO son hojas (solo reciben); el lazo sobre TABLA reúne la diagonal de transformación y control. Diseñar un nodo es elegir dónde poner su flecha en este mapa.

5.2 Por qué los mundos guían el diseño

El mapa de mundos convierte “diseñar un nodo rico en interacciones” en una instrucción operativa. Un nodo es rico cuando su flecha cae en un *cruce de alto tráfico*: su origen es un mundo que muchas familias producen (un destino popular) y su destino es un mundo que muchas familias consumen (un origen popular). TABLA es el cruce por excelencia; un nodo que atraviesa TABLA en cualquiera de sus dos puntas hereda, sin esfuerzo, la posibilidad de componerse con todo el resto del catálogo.

El mapa también hace visible el *antipatrón* de un solo vistazo. Un nodo cuya flecha va de un MUNDO EXÓTICO a otro MUNDO EXÓTICO —un mundo que ningún otro nodo produce, hacia un mundo que ningún otro consume— es una isla: por bien que realice su tarea, nada puede alimentarlo y nada puede leer su salida. Cero interacciones posibles.

Propiedad. *Un nodo $f : X \rightarrow Y$ tiene interacciones potenciales no vacías si y solo si existe algún nodo cuyo destino sea X (un proveedor de su entrada) y algún nodo cuyo origen sea Y (un consumidor de su salida). El diseño aislado no puede verificar esta condición porque no mira el resto del mapa; el modelo de mundos la vuelve una comprobación previa a escribir el nodo.*

La regla de diseño se enuncia entonces sin ambigüedad: *al menos una de las dos puntas de la flecha debe apoyarse en un mundo con tráfico*. Si ambas puntas son exóticas, el nodo no debe diseñarse todavía; primero hay que poblar el mundo exótico con nodos que lo produzcan

o lo consuman, o bien reformular la operación para que una punta aterrice en un concentrador. Esta es, exactamente, la deliberación que el diseño aislado omitía y que causaba el ciclo de retrabajo de la Sección 1.

6 La cuarta coordenada: el carril de transporte

Las tres coordenadas anteriores describen la *forma lógica* de la flecha: a qué mundo lleva el dato, cómo cambia su conteo y cómo se ramifica. Ninguna dice nada sobre la *representación física* de lo que viaja por el cable. Esa es una cuarta coordenada, de naturaleza distinta a las tres, y con una consecuencia que la vuelve la más delicada de todas: no es una optimización, es una *frontera de seguridad*.

6.1 Dos carriles para un mismo mundo

Un mismo mundo —TABLA— admite dos maneras de recorrer el cable. En el carril *por filas* (**row**), el modelo histórico, el lote viaja como N elementos individuales, cada uno una estructura de campos que se materializa y se reparsea en cada frontera de nodo. Es lo correcto para casi todo: registros heterogéneos, pocos, sobre los que cada nodo opera de a uno. En el carril *columnar* (**columnar**), el lote viaja como *un* único búfer contiguo de columnas —un **RecordBatch** de Apache Arrow—: cuatro columnas de un millón de píxeles son cuatro arreglos tipados, no un millón de diccionarios. Es el carril de los datos numéricos masivos y homogéneos: retículas de render, soluciones de ecuaciones diferenciales, series de un gráfico.

Definición 6.1 (Carril de transporte). El carril $\lambda(f) \in \{\mathbf{row}, \mathbf{columnar}\}$ de un nodo es la representación física en la que intercambia su lote sobre un mismo mundo. Es ortogonal a la firma de mundo: no cambia *a dónde* apunta la flecha, sino *cómo* está empaquetado lo que la recorre. Un nodo declara su carril; ausente, es **row**.

Que sea ortogonal significa que dos nodos con la misma firma de mundo $\text{TABLA} \rightarrow \text{TABLA}$ pueden vivir en carriles distintos, y que la composición —que las tres coordenadas declaraban lícita en cuanto $\text{dst}(a) = \text{src}(b)$ — esconde ahora un matiz físico: si a produce en un carril y b consume en otro, *alguien tiene que traducir*.

6.2 El puente entre carriles no es gratis

Cuando un productor columnar se conecta a un consumidor por filas, el motor debe *materializar* el búfer Arrow: convertir sus columnas en los N elementos individuales que el consumidor por filas espera. Esta traducción —el *puente* columnar→fila— es el punto donde una cantidad acotada se vuelve una cantidad enorme. Una retícula de píxeles es, en el carril columnar, *un* objeto: un búfer. Cruzada al carril por filas, es un millón de objetos.

6.3 Por qué es una frontera de seguridad y no una optimización

Aquí está la diferencia esencial con las tres primeras coordenadas. Aquéllas son *advisorias*: el motor nunca ejecuta sobre ellas; sirven para que nosotros —y el asistente— razonemos,

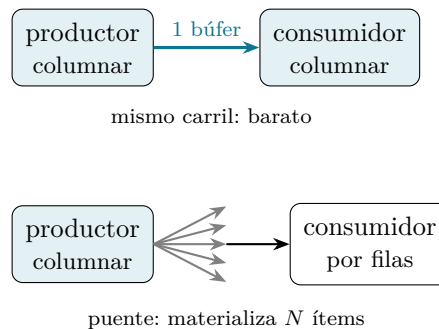


Figura 6: El carril como cuarta coordenada. Columnar→columnar transporta un único búfer de extremo a extremo (barato, acotado). Columnar→fila cruza el *puente*: el búfer se materializa en N elementos individuales, y N puede ser enorme.

pero un flujo que las ignore, a lo sumo, compone mal. El carril no admite ese lujo. El puente columnar→fila es una operación real que consume memoria real, y si no se acota, *un solo nodo basta para tumbar el motor*.

El argumento es directo. Un generador de retícula puede producir, por configuración, millones de píxeles —en el carril columnar, un búfer grande pero manejable—. Conéctese ese generador a un único nodo por filas y el puente materializará millones de elementos individuales en la memoria del motor, de un solo nodo y una sola arista. Peor aún: la coordenada de cardinalidad, que anticipaba las explosiones de lote *dentro* de un carril, no ve esta explosión, porque ocurre *en el cruce entre carriles*, invisible al eje $N:M$. Y la ramificación lo multiplica: un productor columnar que abanica hacia k consumidores por filas paga el puente k veces, aunque cada arista, por separado, parezca inocente.

Observación 6.1 (La vulnerabilidad que la cuarta coordenada expone). *Un modelo que sólo mirara las tres coordenadas lógicas declararí la conexión retícula → nodo por filas como perfectamente válida: $\text{dst} = \text{src} = \text{TABLA}$, cardinalidad fuente hacia preservador, puertos simples. Todo en orden en el papel. Y sin embargo es un ataque de denegación de servicio disparable por cualquier usuario con dos nodos. La frontera entre carriles es, por tanto, el primer lugar del modelo donde la declaración de una coordenada obliga a una defensa en el motor, en vez de limitarse a guiar el diseño.*

6.4 La disciplina de diseño que impone

De reconocer el carril como coordenada se siguen tres reglas, y a diferencia de las de las secciones anteriores, dos de ellas son *obligaciones del motor*, no consejos al diseñador.

1. **Mantener columnar de extremo a extremo lo pesado.** Un pipeline numérico masivo —retícula, cómputo, colorear, rasterizar— debe permanecer en el carril columnar en toda su extensión: un único búfer que fluye sin materializarse jamás. Las conexiones columnar↔columnar son las baratas, y el diseño debe favorecerlas para el volumen.
2. **Acotar todo cruce columnar→fila.** Cada puente debe tener un tope duro: por encima de cierto número de filas materializadas, el nodo falla de forma explícita en lugar de

agotar la memoria. El tope es una *invariante de seguridad*, no un parámetro ajustable que se pueda desactivar.

3. **Acotar también el agregado.** Como la ramificación multiplica el costo, no basta con topar cada arista: hay que topar la *suma* de lo que todos los puentes de un grafo materializarían, para que un productor que abanica a muchos consumidores no burle el tope por-arista repartiéndose.

A esto se añade una obligación sobre el canal de *observabilidad*: la traza que el motor emite hacia el editor no debe crecer con el volumen de elementos, sino mantenerse acotada y coalescente, para que un productor rápido no inunde el canal aunque respete los topes de memoria. Es la misma preocupación —que ninguna cantidad declarada por el usuario se traduzca en un costo ilimitado— vista desde el transporte en lugar de la memoria.

Propiedad. *El carril debe declararse como cuarta coordenada precisamente porque su frontera es peligrosa. Sólo si cada nodo expone su carril pueden el editor, el validador y el asistente distinguir las conexiones baratas (mismo carril) de las que cruzan el puente (y están topadas), y sólo entonces puede el motor situar sus defensas —tope por-cruce, cota al productor, presupuesto agregado— exactamente en las aristas que las necesitan. Las tres primeras coordenadas nos enseñan a componer; la cuarta nos obliga a protegernos.*

7 Consecuencia: un asistente de IA que compone mejor

7.1 De la generación libre a la búsqueda tipada

La construcción de un flujo de trabajo por un asistente de lenguaje es, sin un modelo, un ejercicio de generación libre: el asistente lee las descripciones en lenguaje natural de los nodos, adivina cuáles encajan y produce un grafo cuya corrección solo se comprueba al ejecutarlo. Los errores típicos —conectar la salida de un nodo con una entrada que no puede consumirla, encadenar dos expansores que hacen estallar el lote, dejar un nodo sin fuente que lo alimente— son exactamente los que el modelo de tres coordenadas caracteriza.

Expuestas al asistente —como anotaciones del catálogo que él lee, sin cambio alguno en los nodos—, las coordenadas convierten esa generación libre en una *búsqueda tipada*. Construir un flujo que lleve del mundo X al mundo Y pasa a ser hallar un *camino de flechas componibles* en el mapa de mundos: una búsqueda en un grafo, donde cada paso respeta $\text{dst} = \text{src}$ y donde la cardinalidad acumulada advierte de las explosiones y de las conexiones degeneradas. La tarea deja de ser “inventar” un grafo y pasa a ser “encontrar” uno correcto por composición.

7.2 Por qué basta un modelo pequeño

Este cambio de régimen tiene una consecuencia notable sobre el tamaño de modelo necesario. La generación libre exige que el modelo lleve dentro, de forma implícita, todo el conocimiento de cómo encajan los nodos, y que lo aplique sin equivocarse; es una carga que solo los modelos grandes soportan con soltura. La búsqueda tipada externaliza ese

conocimiento al catálogo: las coordenadas son una *gramática composicional* explícita, y al modelo le queda la tarea, mucho más liviana, de recorrer un espacio ya estructurado.

Propiedad. *Con las tres coordenadas expuestas en el catálogo, la corrección composicional de un flujo —compatibilidad de mundos, ausencia de explosión cardinal, ausencia de nodos aislados— es verificable por reglas locales sobre cada arista. Un asistente que respete esas reglas produce flujos componibles con independencia de su tamaño; el modelo grande deja de ser un requisito para la corrección y pasa a serlo solo para la ambición del diseño.*

En otras palabras: incluso un modelo pequeño puede construir un excelente flujo de trabajo, porque las tres coordenadas le indican cuáles son las mejores conexiones posibles y le vedan las imposibles. El modelo que empezó como una disciplina para que *nosotros* diseñáramos mejor los nodos resulta ser, sin cambios, la disciplina que permite a una máquina *ensamblarlos* mejor. Es la misma gramática, leída desde los dos lados.

8 Conclusiones

Este documento partió de una dificultad concreta —el diseño aislado de nodos y su ciclo de retrabajo— y de un intento honesto pero fallido de remediarlo: definir las familias como clases de equivalencia de una relación de no-conexión. El fracaso de ese intento en la transitividad no fue un callejón sin salida, sino la señal de que el objeto estaba mal elegido. La teoría de conjuntos, que había inspirado el intento, ofreció también la salida: fundar la equivalencia sobre el núcleo de una función en lugar de sobre una relación postulada a mano.

Esa función es la *firma* del nodo, y el nodo mismo dejó de ser un punto para ser una *flecha* entre mundos de datos. Tres coordenadas —la firma de mundo, que fija origen y destino; la cardinalidad; y la aridez de puertos que gobierna la ramificación— bastan para ubicar cualquier nodo en el ecosistema, clasifican desde un **chart** hasta un **switch**, y recuperan como teorema la intuición que la relación fallida quería como definición. Cada coordenada se justificó por el colapso que resolvía y que la anterior no podía.

A las tres coordenadas componibles se suma una cuarta, de otra índole: el *carril de transporte*. Mientras las tres primeras son advisorias —guían el diseño pero el motor no ejecuta sobre ellas—, el carril marca una frontera que el motor *debe* custodiar: el puente entre el envío columnar y el envío por filas materializa un lote acotado en N elementos, y sin un tope —por cruce, sobre el tamaño del productor y sobre el agregado del grafo— un único nodo bastaría para tumbar el sistema. Es el punto donde el modelo deja de ser sólo una disciplina de diseño y se vuelve, además, una disciplina de defensa.

El valor del modelo es, ante todo, metodológico: transforma “¿qué hace este nodo?” en “¿dónde va su flecha, y ese cruce tiene tráfico?”, y con ello convierte el diseño rico en interacciones de un hallazgo tardío y empírico en una comprobación previa. Que la misma estructura, expuesta a un asistente de lenguaje, convierta la construcción de flujos en una búsqueda tipada al alcance de modelos pequeños, es la confirmación de que las coordenadas capturan algo real sobre cómo se componen los nodos —y no meramente sobre cómo los nombramos—. El modelo no restringe el motor ni toca el catálogo: solo nos enseña a mirarlo. Pero esa mirada es, precisamente, lo que faltaba.

Referencias

- [1] K. Hrbacek and T. Jech. *Introduction to Set Theory*, 3rd ed. Marcel Dekker, New York, 1999. (Relaciones de equivalencia, particiones y el núcleo de una función: cap. 2.)
- [2] S. Mac Lane. *Categories for the Working Mathematician*, 2nd ed. Springer, 1998. (El nodo como morfismo y la conexión como composición de flechas.)
- [3] B. C. Pierce. *Basic Category Theory for Computer Scientists*. MIT Press, 1991.
- [4] Mauricio Daniel Klainbard. *Synapse: un modelo de ejecución con cómputo en el flujo*. Documento técnico, 2026. (Modelo de streaming, cardinalidad de consumo y rompedores de tubería.)